

Fyr Data/Code Placement and Linking

The purpose of this document is to provide a specification for the algorithm implemented by *Fyr* for placing code and data and linking them together.

```
1: type Offset = Int
2:
3: function IN-RANGE(Offset me, Offset data)
4:   return  $me < data \wedge (me/4 + 256 \geq data/4)$ 
5:
6: procedure PLACE-BLOCK-DATA(DLL layout, Function func, Block block, Slot slot)
7:   ld_instrs  $\leftarrow$  list ()
8:   for instr in block.instructions do
9:     if instr.is_load then
10:       ld_instrs.prepend(instr)
11:
12:   i  $\leftarrow$  ld_instrs.length - 1
13:   while i  $\geq$  0 do
14:      $\triangleright$  Calculate the offset of the first byte after the block
15:     beoff  $\leftarrow$  slot.offset + slot.size + block.branch.size
16:     if beoff mod 4 > 0 then
17:        $\triangleright$  If the byte is not word aligned, we need to add a null padding slot
18:        $\triangleright$  after the function slot.
19:       padding  $\leftarrow$  beoff + (beoff mod 4 - 1)
20:       slot.add_null_padding_to_right(padding)
21:       slot  $\leftarrow$  slot.right
22:       doff  $\leftarrow$  slot.offset
23:     else
24:       doff  $\leftarrow$  beoff
25:
26:     slot_map  $\leftarrow$  {}
27:     longest_reach  $\leftarrow$  null
28:     while i  $\geq$  0 do
29:       instr  $\leftarrow$  ld_instrs[i]
30:       existing  $\leftarrow$  slot_map[instr.val]
31:       if existing  $\neq$  null  $\wedge$  IN-RANGE(instr.offset, existing.offset) then
32:
33:   function FIBONACCI(n)
34:     data_adjacent_slot  $\leftarrow$  next multiple of 4
35:     if n  $\leq$  1 then
36:       return 1
37:     else
38:       return FIBONACCI(n - 1) + FIBONACCI(n - 2)
```